# Typst Presentation Examples: Complete Feature Showcase

Your Name[1]

[1]Your Institution

March 30, 2026

LOGO Placeholder

# Basic Text Formatting

**Bold text** for emphasis

*Italic text* for subtle emphasis

`Inline code` for technical terms

Colored text in red

Larger text

Smaller text

Normal paragraph text with automatic line wrapping and spacing.

# Lists and Bullets

## Unordered list:

- First item
- Second item
  - Nested item
  - Another nested item
- Third item

## Numbered list:

1. First step
2. Second step
3. Third step

## Mixed list:

- Item with **bold**
- Item with *italic*
- Item with `code`

# Inline Equations

You can write inline math like $x^2 + y^2 = z^2$ within text.

The formula $E = mc^2$ is Einstein's famous equation.

Greek letters: $\alpha, \beta, \gamma, \Delta, \Sigma, \Omega$

Subscripts and superscripts: $x_1, x_2, x^2, x_i^j$

Fractions: $\frac{1}{2}, \frac{a+b}{c+d}$

# Display Equations

## Simple equation:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

## Summation:

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

## Integration:

$$\int_0^\infty e^{-x^2}\,\mathrm{d}x = \frac{\sqrt{\pi}}{2}$$

## Matrix:

# Display Equations

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

# Complex Equations

## System of equations:

$$\begin{cases} x + y = 5 \\ 2x - y = 1 \end{cases}$$

## Aligned equations:

$$\begin{aligned} f(x) &= x^2 + 2x + 1 \\ &= (x + 1)^2 \end{aligned}$$

## Probability:

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

## Set notation:

# Complex Equations

$$A = \{x \in \mathbb{R} \mid x > 0\}$$

# Simple Table

| Name | Value | Unit |
|---|---|---|
| Speed | 100 | km/h |
| Distance | 50 | km |
| Time | 0.5 | h |

# Styled Table

| Metric | Before | After |
|---|---|---|
| Accuracy | 85% | 92% |
| Speed | 10s | 2s |
| Memory | 500MB | 200MB |

# Complex Table

| Method | Accuracy | Speed | Memory | Score |
|---|---|---|---|---|
| Baseline | 80% | Fast | Low | 7.5 |
| Method A | 85% | Medium | Medium | 8.0 |
| Method B | 92% | Slow | High | 8.5 |
| Our Method | **95%** | Fast | Low | **9.2** |

# Python Code - Styled

```python
def extract_features(data):
    """Extract features from accident data"""
    features = []
    for case in data:
        features.append({
            'severity': case['ISS'],
            'vehicles': len(case['vehicles']),
            'weather': case['environment']['weather']
        })
    return features
```

# Python Code - Dark Theme

```python
def extract_features(data):
    """Extract features from accident data"""
    features = []
    for case in data:
        features.append({
            'severity': case['ISS'],
            'vehicles': len(case['vehicles']),
        })
    return features
```

# Cypher Query - Styled

```
// Find causal chains
MATCH (a:Accident)-[:CAUSED_BY]->(f:Factor)
WHERE f.type = "distraction"
RETURN a.case_id, f.description
ORDER BY a.severity DESC
LIMIT 10
```

# Console Output - Terminal Style

```
$ python extract.py --input data/ciren/

Processing case c011... ✓
Processing case c012... ✓
Processing case c013... ✓

Summary:
- Total cases: 25
- Successful: 25
- Failed: 0
- Time: 2m 15s
```

# Console Output - Success/Error

```
$ npm run test
✓ Test suite passed (25/25)
✗ 2 warnings found
⚠ Deprecated function used in line 42
```

# JSON Data - Formatted

```
{
  "case_id": "c011",
  "year": 2017,
  "vehicles": 2,
  "severity": {
    "ISS": 24,
    "MAIS": 4
  },
  "environment": {
    "weather": "clear",
    "lighting": "dark"
  }
}
```

# SQL Query - Styled

```sql
SELECT
  case_id,
  severity,
  COUNT(*) as vehicle_count
FROM accidents
WHERE year = 2017
GROUP BY case_id, severity
ORDER BY severity DESC
LIMIT 10;
```

# Bash Script - Terminal

```bash
#!/bin/bash

# Process all CIREN cases
for case in data/ciren/*.xlsx; do
  echo "Processing $case..."
  python extract.py "$case"
done

echo "Done!"
```

# Code with Line Numbers

```
def process_case(case_id):
    data = load_data(case_id)
    features = extract_features(data)
    graph = build_knowledge_graph(features)
    scenarios = generate_scenarios(graph)
    return scenarios
```

# Side-by-Side Code Comparison - GitHub Style

**Before (Removed)**

```
def slow_process(data):
result = 0
for i in range(len(data)):
for j in range(len(data[i])):
result += data[i][j]
return result
```

**After (Added)**

```
def fast_process(data):
import numpy as np
return np.sum(data)
```

**Performance:** 10× faster with NumPy vectorization

# GitHub-Style Unified Diff

```
@@@ -1,8 +1,5 @@@
def extract_features(report):
context = parse_context(report)
- actors = []

- for section in report.sections:

- actors.extend(find_actors(section))

1. actors = extract_actors(report)

factors = identify_factors(context, actors)
return {'context': context, 'actors': actors}
```

**Changes:** Simplified actor extraction logic (3 lines $\longrightarrow$ 1 line)

# GitHub Diff with Line Numbers

```diff
…   …   src/extraction/pipeline.py
15  15  class ExtractionPipeline:
16      - def __init__(self, config_path):
17      - with open(config_path) as f:
18      - self.config = json.load(f)
    16  + def __init__(self, config: dict):
    17  + self.config = config
19  18   self.stages = []
```

**Refactoring:** Accept config dict instead of file path for better testability

# Single Image

# Single Image

Caption: System architecture diagram

# Multiple Images

# Multiple Images

Left: Before | Right: After

# Image with Text

**Key Points:**

- Feature extraction
- Data processing
- Knowledge graph
- Analysis results

# Bar Chart (ASCII)

```
Accuracy Comparison:

Baseline     ▇▇▇▇▇▇▇▇▇▒▒▒▒▒ 80%
Method A     ▇▇▇▇▇▇▇▇▇▇▒▒▒ 85%
Method B     ▇▇▇▇▇▇▇▇▇▇▇▇ 92%
Our Method   ▇▇▇▇▇▇▇▇▇▇▇▇▇ 95%

0%         50%         100%
```

# Metrics Dashboard

| 92.3% | 10× | 25 |
|---|---|---|
| Accuracy | Faster | Cases |

# Progress Indicators

## Task Completion:

# Comparison Matrix

| Feature | Ours | Method A | Method B |
|---|---|---|---|
| Real Data | ✓ | ✗ | ✗ |
| Ontology | ✓ | ✗ | ✓ |
| KG Support | ✓ | ✗ | ✗ |
| ODD-Aware | ✓ | ✗ | ✗ |
| Causal Analysis | ✓ | ✗ | ✗ |

# Two-Column Layout

## Left Column:

- Point 1
- Point 2
- Point 3

$$x^2 + y^2 = r^2$$

## Right Column:

- Point A
- Point B
- Point C

$$E = mc^2$$

# Three-Column Layout

| Column 1 | Column 2 | Column 3 |
| --- | --- | --- |
| Content here | Content here | Content here |

# Mixed Content

## Main Content:

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

$$f(x) = \int_0^x t^2 \, \mathrm{d}t$$

```
def process(data):
    return data * 2
```

LOGO Placeholder

## Metrics:

- 92% accuracy
- 10× faster
- 25 cases

# Theorem Box - LaTeX Style

**Theorem (Causal Chain Theorem)**

For any accident $A$ in knowledge graph $G = (V, E)$, there exists a causal chain $C = \{f_1, f_2, ..., f_n\}$ where $f_i \in F$ (factors) such that:

$$A = \prod_{i=1}^{n} f_i$$

*Proof.* By construction of the knowledge graph, each accident node is connected to at least one causal factor through CAUSED_BY relationships. □

# Definition Box - LaTeX Style

## Definition (Operational Design Domain (ODD))

An ODD is defined as a 5-tuple $O = (R, E, D, C, G)$ where:

- $R$: Roadway characteristics
- $E$: Environmental conditions
- $D$: Dynamic elements
- $C$: Connectivity requirements
- $G$: Geographic boundaries

# Example Box - LaTeX Style

## Example (CIREN Case Analysis)

Consider case c011 with the following properties:

- ISS score: 24 (serious injury)
- Vehicles involved: 2
- Collision type: Head-on frontal
- Primary factor: Lane departure

This case demonstrates the importance of lane-keeping systems in AV design.

# Lemma and Corollary

## Lemma (Factor Independence)

Causal factors in the knowledge graph are not necessarily independent.

## Corollary

Therefore, risk assessment must consider factor interactions and synergistic effects.

# Remark Box - Minimal Style

**Remark 1:** The extraction pipeline processes cases sequentially to maintain data consistency and enable incremental updates.

**Note:** All statistical analyses use a significance level of $\alpha = 0.05$ unless otherwise specified.

# Important Notes - Academic Style

**Important:** Data privacy considerations require anonymization of all personally identifiable information before processing.

**Assumption:** We assume that accident reports are complete and accurate as recorded by NHTSA investigators.

# Quote Block - LaTeX Style

> *"Knowledge graphs provide a powerful framework for representing complex relationships in accident data, enabling sophisticated causal analysis and pattern discovery."*
>
> — Zhang et al. (2022)

# Enumerated Items - LaTeX Style

## Research Contributions:

1. Development of 5-layer accident ontology
2. Implementation of 6-stage extraction pipeline
3. Design of multi-model database architecture
4. Generation of 21 challenge scenarios
5. Integration of ODD framework with accident analysis

# Nested Lists - Academic Style

## System Architecture:

- Input Layer
  - CIREN accident reports
  - Research literature
  - Domain knowledge
- Processing Layer
  - Stage 1: Context extraction
  - Stage 2: Actor identification
  - Stage 3: Causal analysis
- Storage Layer
  - Neo4j graph database
  - PostgreSQL relational database
  - RDF triple store

# Footnotes and References

The knowledge graph approach[1] enables systematic accident analysis.

Our extraction pipeline achieves 92.3% accuracy[2], significantly outperforming manual methods.

The ODD framework[3] provides structured scenario specification.

[1] Zhang et al., "Analysis of Traffic Accident Based on Knowledge Graph," Journal of Advanced Transportation, 2022.

[2] Comparable to Text2Scenario (Cai et al., 2025) which reported 92.3% parsing accuracy.

[3] Based on JAMA's Automated Driving Safety Evaluation Framework Version 3.0.

# Flow Diagram

```
┌──────────────┐
│  Raw Data    │
└──────────────┘
       │
       ▼
┌──────────────┐
│ Extraction   │
└──────────────┘
       │
       ▼
┌──────────────┐
│ Knowledge    │
│   Graph      │
└──────────────┘
       │
       ▼
```

# Flow Diagram

```
│  Analysis   │
└──────────────┘
```

# Architecture Diagram

```
┌──────────────────────────────────────┐
│            Input Layer               │
│   ┌──────────┐   ┌──────────┐        │
│   │ CIREN    │   │ Papers   │        │
│   └──────────┘   └──────────┘        │
└──────────────────────────────────────┘
                 │
                 ▼
┌──────────────────────────────────────┐
│        Processing Layer              │
│   ┌──────────────────────────┐       │
│   │  6-Stage Extraction      │       │
│   └──────────────────────────┘       │
└──────────────────────────────────────┘
                 │
                 ▼
┌──────────────────────────────────────┐
│         Storage Layer                │
```

# Architecture Diagram

```
│  Neo4j │ PostgreSQL │ RDF          │
└──────────────────────────────────────┘
```

# Knowledge graphs enable systematic accident analysis

# Emphasis Techniques

## Main Finding

92.3% accuracy achieved

10× faster than manual processing

# Additional Resources

## Documentation:

- Typst docs: https://typst.app/docs/
- Touying: https://touying-typ.github.io/

## Source Code:

- GitHub: github.com/your-repo

## Contact:

- Email: your.email@institution.edu

# JSON - LaTeX Listing Style

```
{
  "case_id": "c011",
  "year": 2017,
  "vehicles": 2,
  "severity": {
    "ISS": 24,
    "MAIS": 4
  }
}
```

**Listing**: Listing 1: CIREN case data structure

# SQL - LaTeX Listing Style

```sql
SELECT case_id, severity, COUNT(*) as count
FROM accidents
WHERE year = 2017
GROUP BY case_id, severity
ORDER BY severity DESC
LIMIT 10;
```

**Listing**: Listing 2: SQL query for severity analysis

# Algorithm Pseudocode

**Algorithm 1:** Knowledge Graph Construction

**Input:** Reports $R = \{r_1, ..., r_n\}$

**Output:** Graph $G = (V, E)$

1: **for each** $r_i \in R$ **do**

2: entities $\leftarrow$ Extract$(r_i)$

3: $V \leftarrow V \cup$ entities

4: **end for**

5: **return** $G$

**Figure**: Algorithm 1: KG construction

# Inline Code Examples

Use `extract_features()` to process data.

Query with `MATCH (a:Accident)` in Neo4j.

**Note:** Code follows PEP 8 guidelines.